SenEvent

Accueil Nouvel evenement atou@... Se deconnecter

DAWM · BLOC BACKEND 2/2 · M1 RETEL — DMI/FST/UCAD

Lab 6 — CRUD Supabase depuis React

Rapport de réalisation : le mock JSON disparaît au profit de vraies requêtes PostgreSQL. Lecture, création, inscription et suppression passent par `supabase-js`, avec Row Level Security qui refuse en direct ce que l'interface se contente de cacher. La persistance devient réelle : F5 ne perd plus rien.

tag : v0.6-crud | SELECT · INSERT · DELETE | Jointure profiles | Storage | RLS vérifiée sur la vraie base | Build ✓ 0 erreur

01 Le remplacement : du fichier statique à la base

Depuis le Lab 1, le catalogue venait d'un fichier figé dans `public/`. Le Lab 6 coupe ce dernier fil. À gauche le code des Labs 1–5, à droite celui d'aujourd'hui : même rôle, mais l'un lit un fichier, l'autre interroge PostgreSQL.

AVANT — APP.JSX

```
// Labs 1–5 : lecture seule, figee dans un fichier statique
const reponse = await fetch("/evenements.json");
if (!reponse.ok) throw new Error(`Erreur HTTP ${reponse.status}`);
const data = await reponse.json();
```

APRÈS — APP.JSX

```
const charger = async () => {
  setChargement(true);
  setErreur(null);
  const { data, error } = await supabase
    .from("evenements")
    .select("*, profiles(nom)")
    .order("date_debut", { ascending: true });
  if (error) {
    setErreur(error.message);
  } else {
    setEvenements(data);
  }
  setChargement(false);
};
```

Trois différences de fond. Il n'y a plus de `try / catch` : Supabase ne lève pas d'exception, il renvoie `{ data, error }`, et l'on teste `error` avant de toucher `data`. Le tri par `date_debut` est délégué à PostgreSQL plutôt qu'au client. Et `profiles(nom)` ramène le nom de l'organisateur dans la même requête — c'est l'étape 5, obtenue en sept caractères.

Vérification imposée par l'énoncé : `grep -rn "evenements.json" apps/web/src` ne renvoie plus aucune ligne.

02 Les six opérations câblées

Le CRUD complet, réparti sur quatre fichiers. Chaque appel suit le même contrat : on vérifie `error`, puis on utilise `data`.

CRUD	APPEL SUPABASE	RÔLE DANS SÉNEVENT	FICHIER
Read	`.from('evenements').select('*, profiles(nom)')`	Catalogue trie par date, avec le nom de l'organisateur	`App.jsx`
Create	`.from('evenements').insert({...})`	Creation d'un evenement, organisateur_id = auth.uid()	`NouvelEvenement.jsx`
Create	`.from('inscriptions').insert({...})`	S'inscrire a un evenement	`BoutonInscription.jsx`
Delete	`.from('inscriptions').delete().eq(...).eq(...)`	Se desinscrire	`BoutonInscription.jsx`
Delete	`.from('evenements').delete().eq('id', x)`	Suppression par l'organisateur seulement	`Detail.jsx`
Storage	`.storage.from('affiches').upload(chemin, fichier)`	Televersement de l'affiche, puis getPublicUrl()	`NouvelEvenement.jsx`

03 Créer un événement, puis re-lire

Le formulaire du Lab 4 gardait sa saisie dans un `useState` que le premier F5 effaçait. Il écrit maintenant en base. On passe `organisateur_id` explicitement, mais ce n'est pas ce qui sécurise : la policy `with check (organisateur_id = auth.uid())` le vérifie en base, et refuserait un identifiant emprunté — ce que la section 05 démontre.

NOUVELEVENEMENT.JSX

```
const { data: { user } } = await supabase.auth.getUser();
if (!user) {
  setErreurServeur("Vous devez etre connecte.");
  setEnCours(false);
  return;
}

const { error } = await supabase.from("evenements").insert({
  titre: titre.trim(),
  categorie,
  lieu_nom: lieu.trim(),
  prix: Number(prix),
  date_debut: new Date().toISOString(),
  image_url,
  organisateur_id: user.id,   // RLS verifie que c'est bien auth.uid()
});

if (error) {
  setErreurServeur(error.message);
  return;
}
await onAjoutReussi();  // re-fetch : la liste redevient ce que la base dit
navigate("/");
```

Après l'écriture, deux stratégies existaient. La **mise à jour optimiste** ajoute la ligne à l'état local sans attendre la base : plus rapide à l'œil, mais il faut défaire en cas d'échec. Le **re-fetch**, retenu ici, rappelle `charger()` : une requête de plus, mais la liste reste exactement ce que la base contient — pas de divergence possible entre l'écran et la vérité. Sur le volume de SénEvent, la simplicité gagne.

04 S'inscrire : deux écritures symétriques

Un même composant porte trois états — non connecté, inscrit, non inscrit — et deux écritures inverses. Au montage, un `SELECT` filtré demande à la base si l'inscription existe déjà : l'état vient de PostgreSQL, jamais d'une supposition du client.

COMPONENTS/BOUTONINSCRIPTION.JSX

```
// Deja inscrit ? Les .eq() se chainent : WHERE ev_id = ... AND user_id = ...
const { data, error } = await supabase
  .from("inscriptions")
  .select("id")
  .eq("evenement_id", evenementId)
  .eq("utilisateur_id", session.user.id);

// S'inscrire = INSERT ; se desinscrire = DELETE cible par les memes filtres
const sInscrire = async () => {
  const { error } = await supabase.from("inscriptions").insert({
    evenement_id: evenementId,
    utilisateur_id: session.user.id,
  });
  if (error) setErreur(error.message); else setInscrit(true);
};

const seDesinscrire = async () => {
  const { error } = await supabase
    .from("inscriptions")
    .delete()
    .eq("evenement_id", evenementId)
    .eq("utilisateur_id", session.user.id);
  if (error) setErreur(error.message); else setInscrit(false);
};
```

Les `.eq()` s'enchaînent en un `AND` : `WHERE evenement_id = 12 AND utilisateur_id = 'abc'`. C'est le motif pour viser une ligne précise avant un `DELETE`. La contrainte `unique (evenement_id, utilisateur_id)` du Lab 5 reste la ceinture : même si un bug tentait un double `INSERT`, PostgreSQL le refuserait — vérifié en section 06.

05 Supprimer : l'UI cache, RLS protège

L'organisateur voit un bouton rouge sur ses propres événements. Les autres ne le voient pas — mais c'est du confort d'interface, pas une sécurité : on n'affiche pas une action impossible.

PAGES/DETAIL.JSX

```
// L'UI cache le bouton : confort, pas securite.
const estOrganisateur =
  session && session.user.id === evenement.organisateur_id;

const supprimer = async () => {
  if (!window.confirm("Supprimer cet evenement ?")) return;
  const { error } = await supabase
    .from("evenements")
    .delete()
    .eq("id", evenement.id);   // RLS refuse si ce n'est pas le proprietaire
  if (error) { setErreurSuppression(error.message); return; }
  await onSupprime();
  navigate("/");
};

{estOrganisateur && (
  <button onClick={supprimer} className={styles.supprimer}>
    Supprimer cet evenement
  </button>
)}
```

Ne jamais compter sur l'UI seule

Cacher un bouton n'empêche rien : la console reste ouverte, et `supabase.from("evenements").delete().eq("id", …)` s'appelle à la main. Ce qui bloque vraiment, c'est `using (organisateur_id = auth.uid())`, évalué par PostgreSQL à chaque ligne. Le `DELETE` d'un compte non propriétaire ne lève pas d'erreur — plus subtil : il réussit en ne touchant **aucune ligne**, parce que RLS a rendu ces lignes invisibles à l'opération. C'est exactement ce que montre le tableau suivant.

06 Ce que la base a réellement répondu

Scénario joué contre le projet Supabase avec deux comptes réels — A (Atou Diop, organisateur) et B (Khady Ndiaye) — et non contre un mock. Chaque ligne est une requête HTTP réelle et sa réponse.

CAS	SCÉNARIO	RÉPONSE DE LA BASE	CONFORME
INSERT autorise	A cree un evenement avec organisateur_id = son uid	`HTTP 201 — evenement #8 cree`	✓
INSERT usurpe refuse	B tente organisateur_id = uid de A	`HTTP 403 — 42501 : violation de la policy with check`	✓
Inscription	B s'inscrit a l'evenement de A	`HTTP 201 — ligne #1 creee`	✓
Double inscription refusee	B reinsere la meme paire (evenement, utilisateur)	`HTTP 409 — 23505 : contrainte unique violee`	✓
DELETE refuse par RLS	B tente de supprimer l'evenement de A	`HTTP 200 — 0 ligne touchee, evenement toujours present`	✓
Upload Storage	A televerse une affiche dans son dossier	`HTTP 200 — objet cree, URL publique obtenue`	✓
Upload usurpe refuse	B televerse dans le dossier de A	`HTTP 400 — policy de chemin : refuse`	✓

Les deux lignes qui comptent sont les refus. L'`INSERT` usurpé est rejeté franchement (violation de `with check`) : on ne peut pas s'inscrire organisateur à la place d'un autre. Le `DELETE`, lui, renvoie un succès en ne supprimant **0 ligne** — l'événement de A est toujours en base après la tentative de B. Deux formes de refus, une seule cause : la politique est évaluée en base, hors d'atteinte du client.

07 Bonus — Storage : la troisième brique Supabase

Après Auth et PostgreSQL, le stockage de fichiers. `image_url` ne pointe plus forcément vers un placeholder : le formulaire accepte une affiche, la téléverse, et récupère son URL publique avant l'`INSERT`.

NOUVELEVENEMENT.JSX — UPLOAD PUIS INSERT

```
let image_url = `https://placehold.co/400x250/1a3a5c/fff?text=${categorie}`;
if (fichier) {
  // Le chemin commence par l'uid : c'est ce qu'exige la policy Storage.
  const chemin = `${user.id}/${Date.now()}_${fichier.name}`;
  const { error: erreurUpload } = await supabase.storage
    .from("affiches")
    .upload(chemin, fichier);
  if (erreurUpload) { setErreurServeur(erreurUpload.message); return; }

  // getPublicUrl est synchrone : il construit l'URL, sans appel reseau.
  const { data: urlData } = supabase.storage
    .from("affiches")
    .getPublicUrl(chemin);
  image_url = urlData.publicUrl;
}
```

MIGRATIONS/006_STORAGE.SQL

```
insert into storage.buckets (id, name, public)
values ('affiches', 'affiches', true)
on conflict (id) do nothing;

-- Lecture pour tous, y compris anonyme (le catalogue est public)
create policy "affiches_select_public"
  on storage.objects for select
  using (bucket_id = 'affiches');

-- Upload authentifie, et uniquement dans son propre dossier "<uid>/..."
create policy "affiches_insert_auth"
  on storage.objects for insert to authenticated
  with check (
    bucket_id = 'affiches'
    and (storage.foldername(name))[1] = auth.uid()::text
  );
```

Le bucket est créé en SQL plutôt qu'à la souris : il rejoint les migrations du Lab 5, et l'infrastructure reste reproductible. Storage expose les fichiers comme des lignes de `storage.objects` — la sécurité s'y écrit donc en RLS, exactement comme sur nos tables. La policy impose que le chemin commence par l'uid de l'auteur : B ne peut pas écrire dans le dossier de A, vérifié en section 06.

08 Un correctif hérité du Lab 5

L'étape 5 affiche le nom de l'organisateur — encore faut-il qu'il existe. Le signup du Lab 5 insérait lui-même la ligne `profiles`, alors que le trigger `on_auth_user_created` l'avait déjà créée : les deux écritures se marchaient dessus et le nom retombait sur le défaut `'Invite'`. Le trigger lit `raw_user_meta_data ->> 'nom'` : c'est donc à `signUp` de le transmettre, et l'insertion manuelle disparaît.

PAGES/AUTH.JSX

```
// Le nom part dans raw_user_meta_data : le trigger on_auth_user_created
// (Lab 5) le lit pour creer la ligne profiles dans la meme transaction.
const { error } = await supabase.auth.signUp({
  email,
  password: motDePasse,
  options: { data: { nom } },
});
```

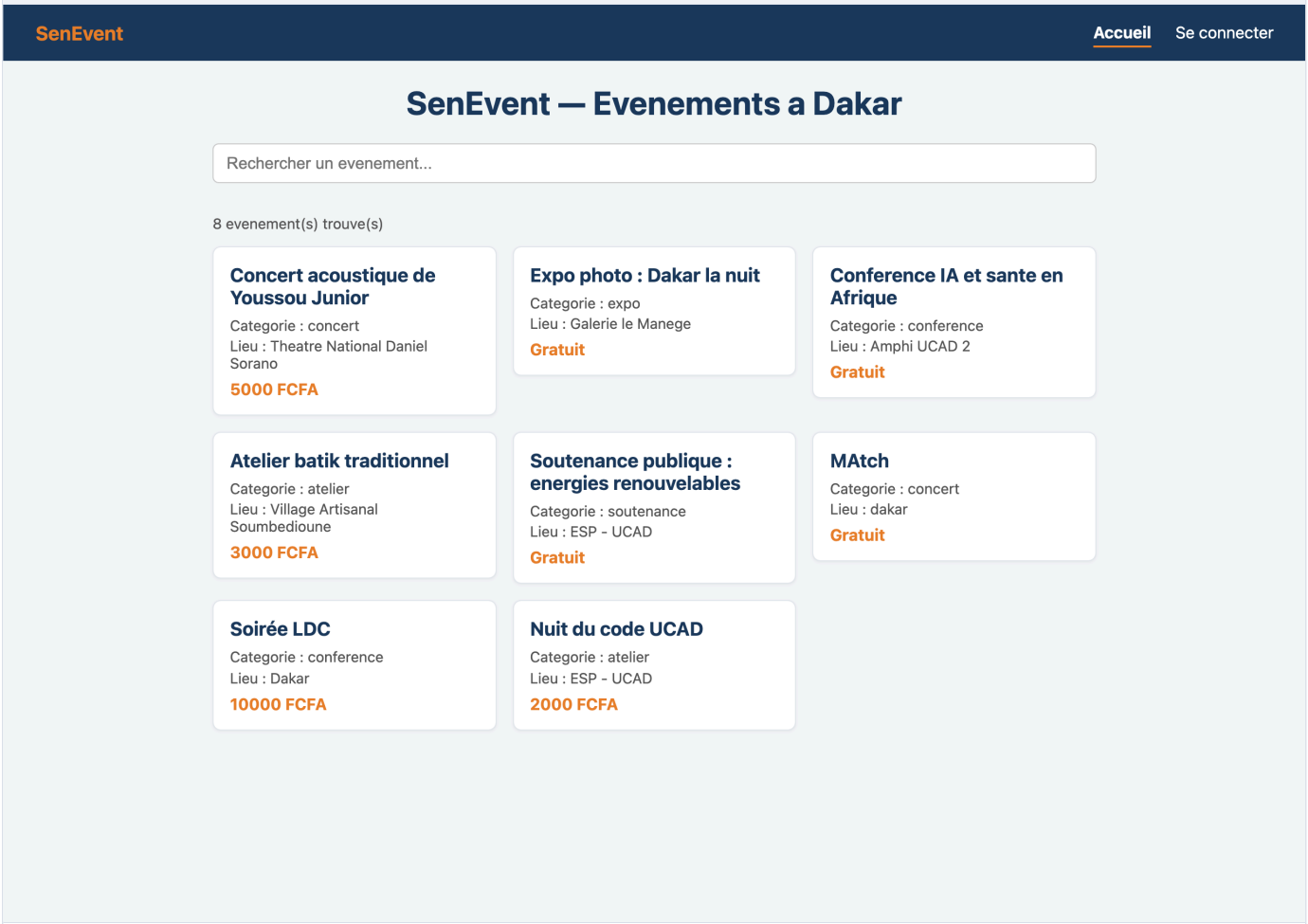
Les profils créés pendant ce lab portent désormais leur vrai nom (`Atou Diop`, `Khady Ndiaye`), et « Organisé par » affiche autre chose que « Invite ». Les 5 événements seed, eux, n'ont pas d'organisateur : leur `profiles` vaut `null` et l'interface retombe sur « Equipe SenEvent ».

09 Le parcours, en situation

Captures prises sur l'application en fonctionnement, avec des sessions réelles et la vraie base : lecture du catalogue, création, inscription, persistance après rechargement, et bascule de l'interface selon le propriétaire.

FIGURE 1 /

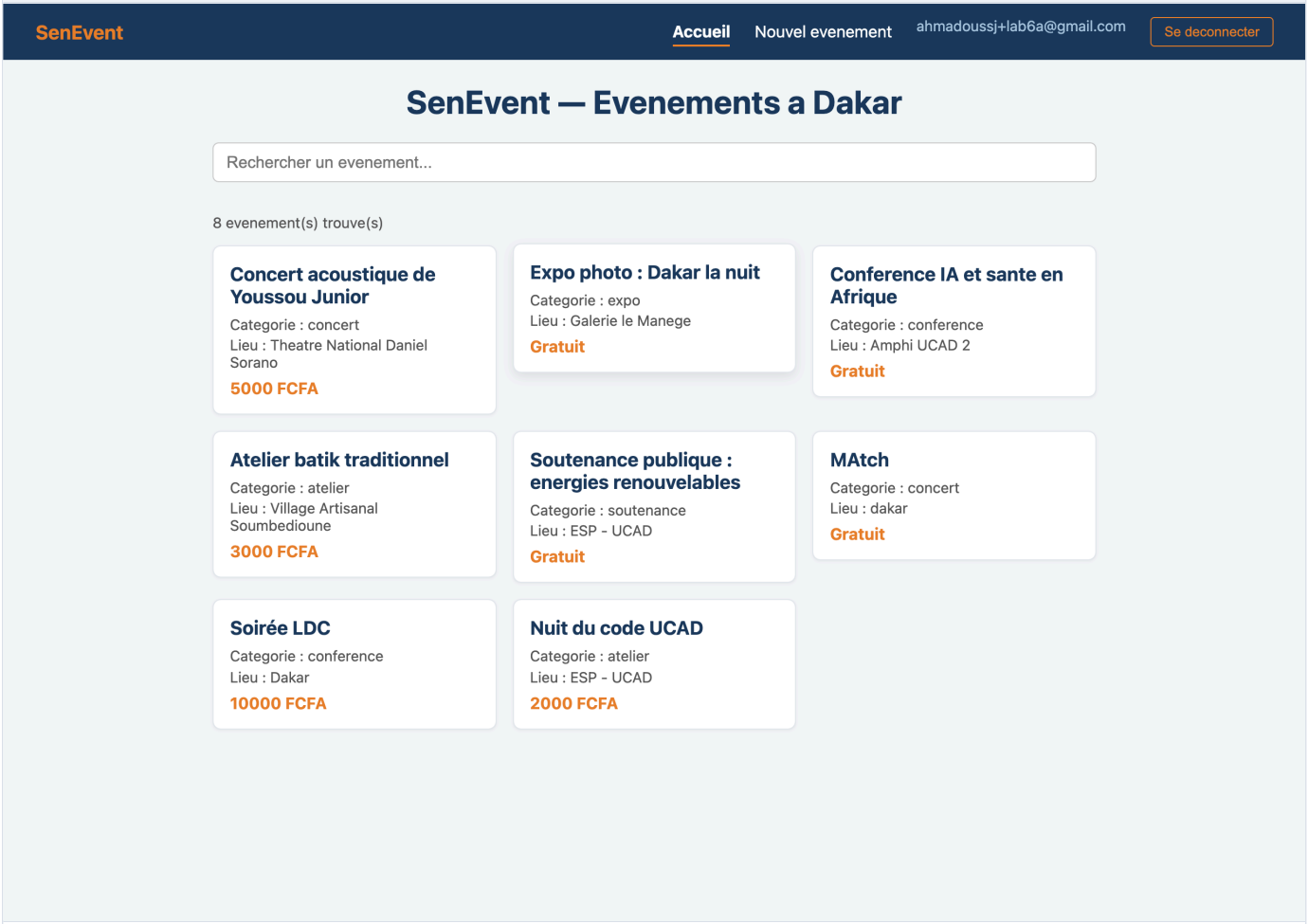
Le catalogue vient de PostgreSQL



Visuellement proche du Lab 5, mais l'onglet Network ne montre plus `evenements.json` : la liste arrive de `siszafptxskyfdedpqsl.supabase.co`, triee par `date_debut` cote base. Les evenements crees pendant le lab s'inserent d'eux-memes dans l'ordre chronologique.

FIGURE 2 /

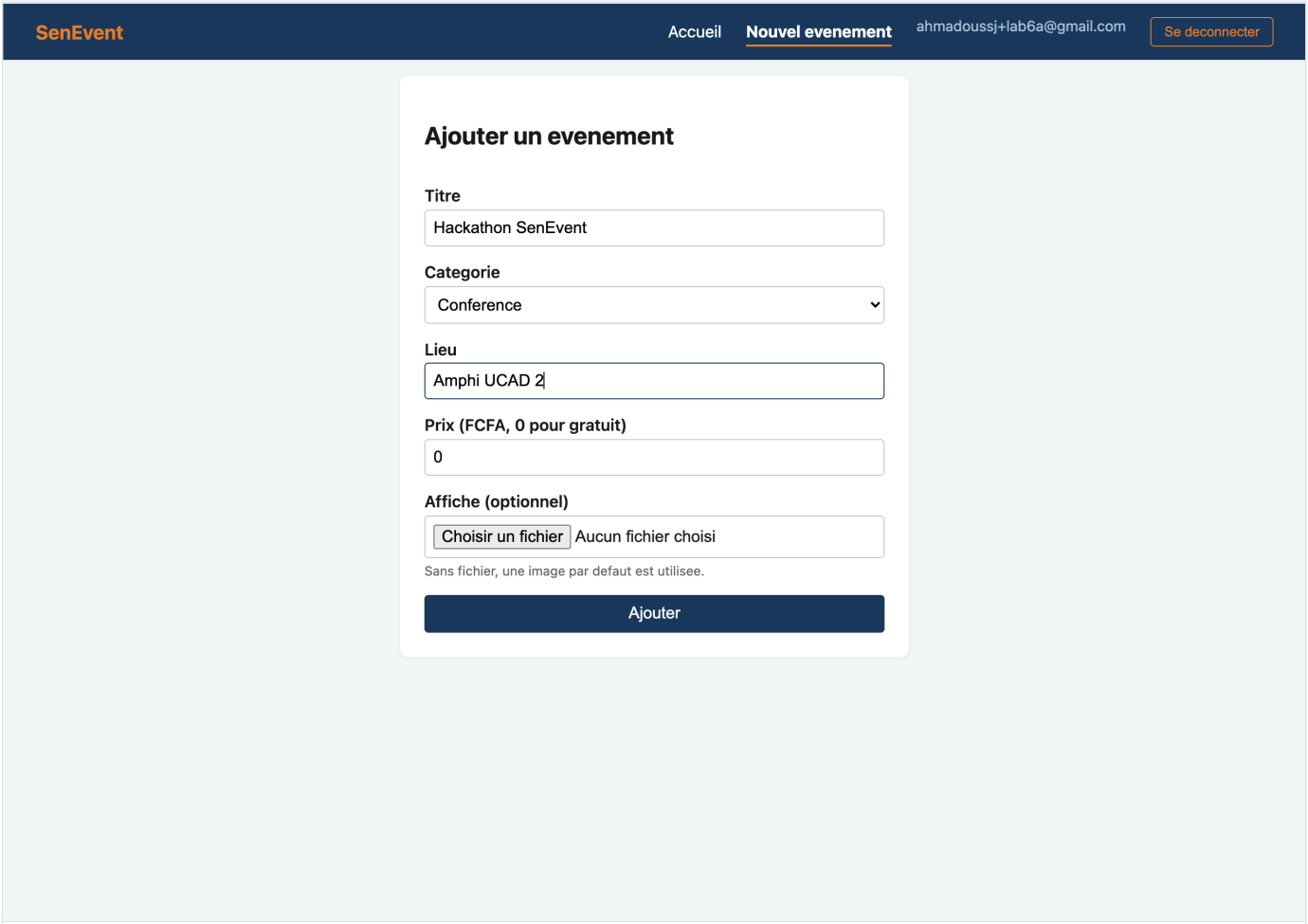
Session reelle, ecritures debloquees



Connexion du compte A (Atou Diop). `onAuthStateChange` met a jour l'etat `session`, la NavBar bascule et « Nouvel evenement » apparait. Sans cette session, la policy `with check` refuserait tout INSERT.

FIGURE 3 `/nouveau`

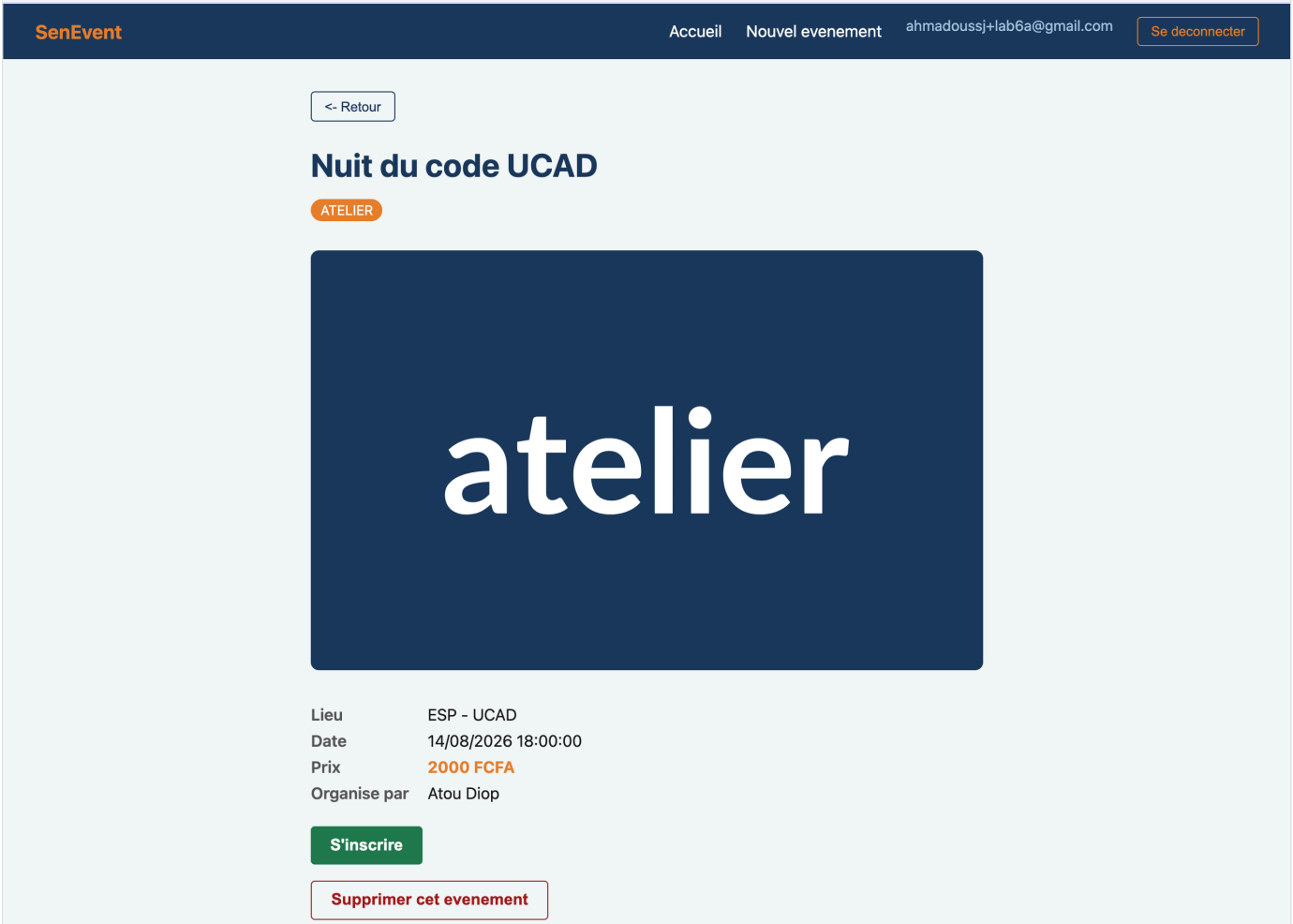
Le formulaire ecrit en base



Les memes champs controles qu'au Lab 4, mais la soumission fait un `.insert()` reel. Le champ fichier alimente le bonus Storage ; sans fichier, un placeholder est utilise. Le bouton se verrouille pendant l'envoi (`enCours`).

FIGURE 4 `/evenement/:id`

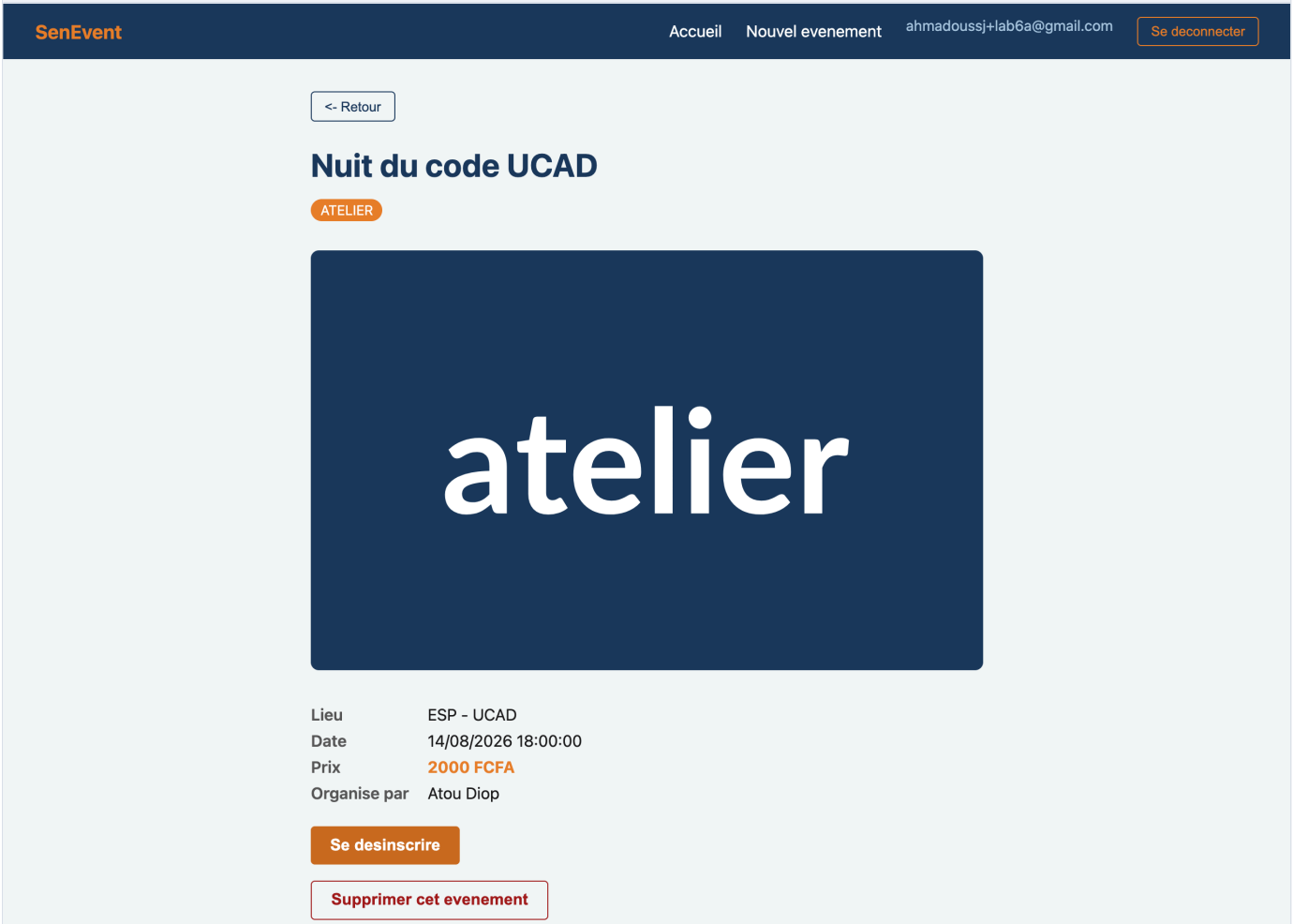
Jointure, inscription, suppression



Trois etapes du lab dans un seul ecran. « Organise par : Atou Diop » vient de la jointure `profiles(nom)`. Le bouton vert ecrit dans `inscriptions`. Le bouton rouge n'apparait que parce que la session est celle de l'organisateur.

FIGURE 5 `/evenement/:id`

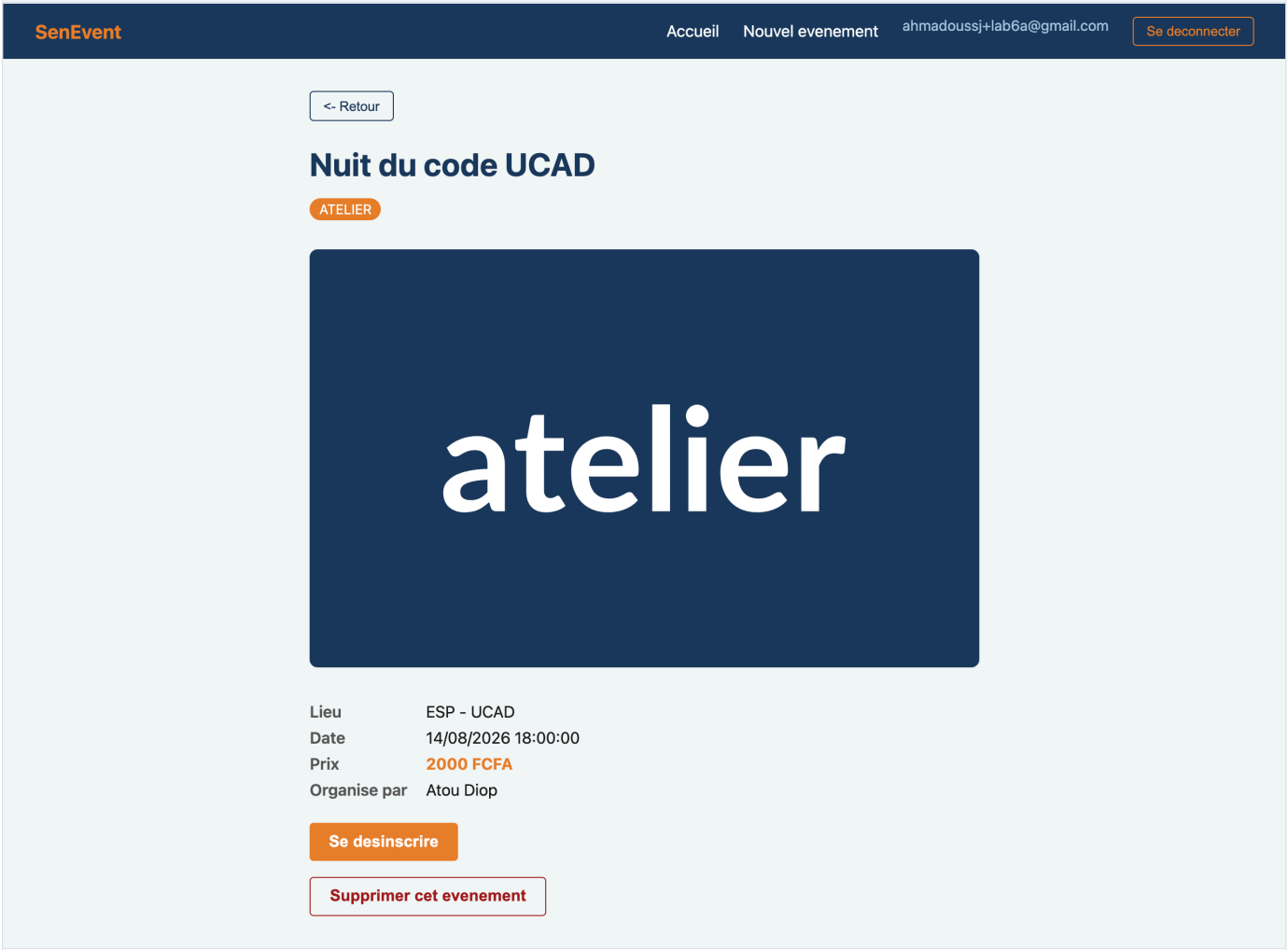
L'inscription bascule le bouton



Le clic declenche un `INSERT` dans `inscriptions` ; l'etat `inscrit` passe a `true` et le bouton devient « Se desinscrire », qui fera le `DELETE` symetrique. La ligne est visible dans le Table Editor.

FIGURE 6 `/evenement/:id`

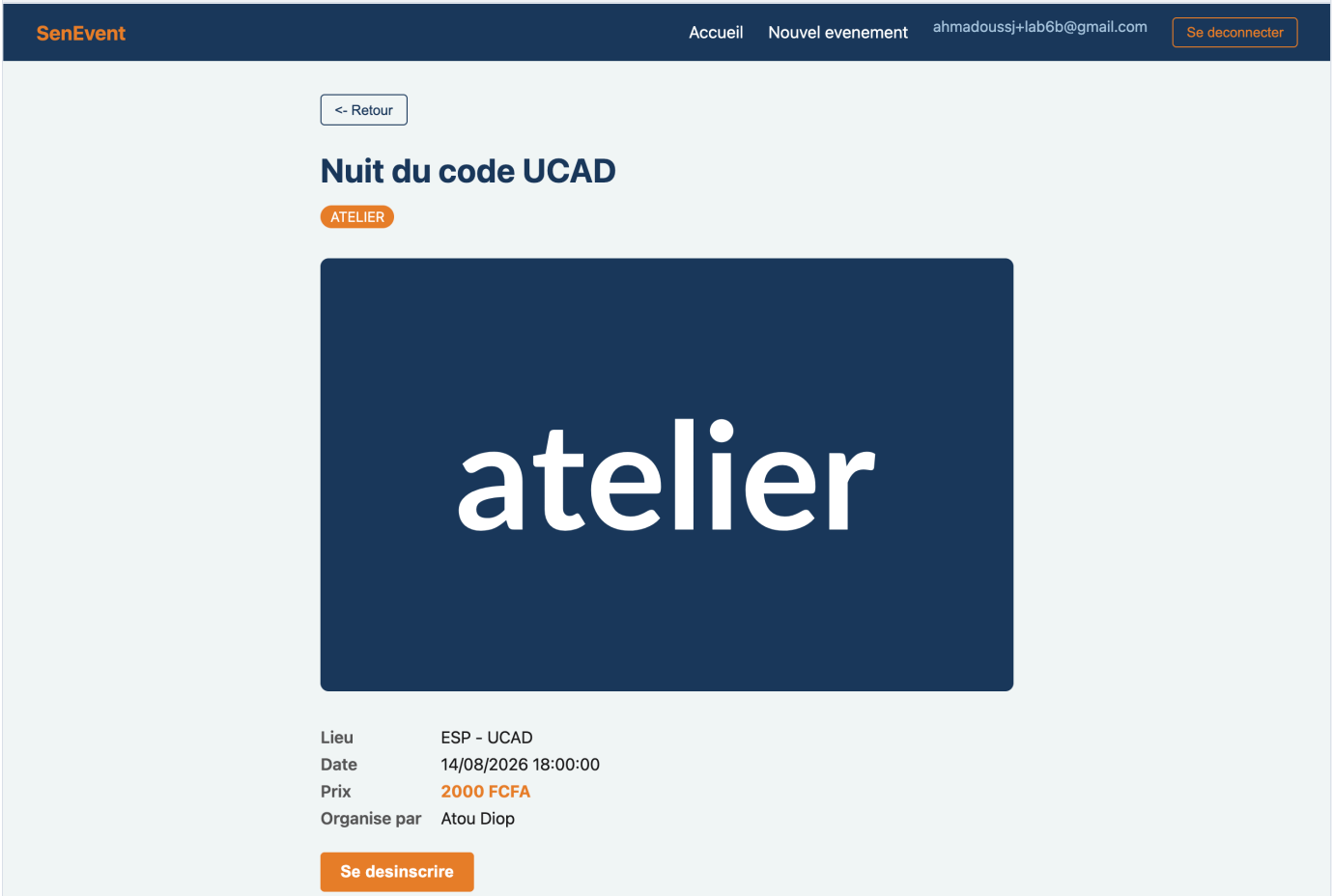
F5 ne perd plus rien



Apres rechargement complet, le bouton reste sur « Se desinscrire ». L'etat n'est pas dans React : il est relu depuis la base au montage. C'est la difference entre le Lab 4 et le Lab 6 en une capture.

FIGURE 7 `/evenement/:id`

Compte B : le bouton disparait



Le meme evenement, vu par le compte B. `session.user.id !== evenement.organisateur_id` : le bouton « Supprimer » n'est pas rendu. L'inscription, elle, reste offerte a tout le monde.

Vers le Lab 7

L'application web est complète : catalogue réel, création, inscription, suppression, upload — tout persiste. Mais toute la logique Supabase est écrite en dur dans les composants web : `.from().select()`, les formes de données, les helpers d'auth. Construire le mobile en React Native (Labs 8–10) obligerait à réécrire la même chose une seconde fois — la duplication qu'on s'est promis d'éviter. Au Lab 7, cette logique part dans `packages/shared` : workspaces npm, TypeScript pour typer les données, et des fonctions réutilisables (`getEvenements()`, `creerEvenement()`) à la place des appels dispersés. Une seule source de vérité, consommée par le web puis par le mobile.